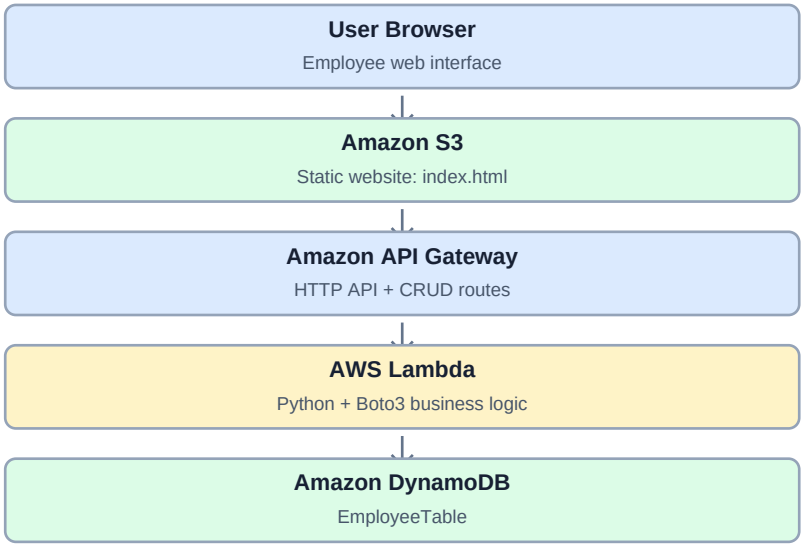
AWS Employee Management Web Application

Complete Step-by-Step Project Documentation



Technology: Amazon S3 • API Gateway HTTP API • AWS Lambda (Python) • DynamoDB • IAM • HTML/CSS/JavaScript

Project status: End-to-end CRUD operations tested through Postman and the S3-hosted frontend.

Prepared as a GitHub portfolio and interview reference

1. Project Overview

This serverless application manages employee records through a web interface. Amazon S3 serves the static frontend. JavaScript calls an Amazon API Gateway HTTP API. API Gateway invokes a Python AWS Lambda function, which performs database operations on Amazon DynamoDB using Boto3. The project was built in AWS region ap-southeast-2 (Asia Pacific – Sydney).

Learning objectives

• Build a serverless web application architecture.
• Configure IAM permissions for Lambda-to-DynamoDB access.
• Implement CRUD operations with Python and Boto3.
• Expose Lambda through API Gateway HTTP API routes.
• Host a static frontend on S3 and configure CORS.
• Test API routes and browser integration.

2. AWS Resources and Configuration

Resource	Configured value
Region	ap-southeast-2 (Sydney)
DynamoDB table	EmployeeTable
Partition key	employeeId (String)
Lambda function	EmployeeManagementFunction
Runtime	Python 3.12
IAM role	Lambda-EmployeeApp-ExecutionRole
API Gateway	EmployeeManagementAPI (HTTP API)
API ID	jndh8i2d68
Stage	$default; auto-deploy enabled
S3 frontend	Static website hosting; index.html

API base URL: https://jndh8i2d68.execute-api.ap-southeast-2.amazonaws.com

3. Architecture and Request Flow

The frontend does not connect directly to DynamoDB. Every data request passes through API Gateway and Lambda. The Lambda execution role grants the function scoped permission to access EmployeeTable. For example, the Add Employee action sends POST /employees with JSON. Lambda processes the request and writes the item to DynamoDB.

4. Step-by-Step AWS Setup

Step 1 — Create DynamoDB table

1. Open AWS Console and select ap-southeast-2.
2. Open DynamoDB → Tables → Create table.
3. Table name: EmployeeTable.
4. Partition key: employeeId; type: String.
5. Create the table and wait for Active status.

Step 2 — Create the Lambda IAM role

1. Open IAM → Roles → Create role.
2. Choose AWS service → Lambda.
3. Attach AWSLambdaBasicExecutionRole for CloudWatch logging.
4. Create and attach a customer-managed policy named EmployeeTable-Lambda-Access.
5. Attach the role to the Lambda function.

The custom policy should grant only the required DynamoDB actions and scope them to the table ARN:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": [
      "dynamodb:PutItem", "dynamodb:GetItem",
      "dynamodb:Scan", "dynamodb:UpdateItem",
      "dynamodb:DeleteItem"
    ],
    "Resource": "arn:aws:dynamodb:ap-southeast-2:YOUR_ACCOUNT_ID:table/EmployeeTable"
  }]
}
```

Replace YOUR_ACCOUNT_ID with the account ID in your own AWS environment. Never upload credentials or secret keys to GitHub.

Step 3 — Create Lambda function

1. Open Lambda → Create function → Author from scratch.
2. Name: EmployeeManagementFunction.
3. Runtime: Python 3.12.
4. Select Lambda-EmployeeApp-ExecutionRole.
5. Create function; paste the code from backend/lambda_function.py; Save and Deploy.

The handler should parse API Gateway HTTP API payload format 2.0, read the HTTP method and employeeId path parameter, perform the requested database operation, and return a JSON response. The project handler also includes Decimal-aware JSON serialization for DynamoDB numeric values.

Step 4 — Implement CRUD logic

Operation	DynamoDB operation	Expected response
Create	PutItem	201 Created
List all	Scan	200 OK
Read one	GetItem	200 OK / 404
Update	UpdateItem	200 OK
Delete	DeleteItem	200 OK / 404

Implementation notes

• Parse the request body as JSON.
• Validate required fields such as employeeId.
• Use conditional expressions to prevent duplicate creates and avoid updating/deleting missing items.
• Return Content-Type: application/json.
• Handle malformed JSON and database exceptions with appropriate HTTP status codes.

Step 5 — Create API Gateway HTTP API

1. Open API Gateway → Create API → HTTP API.
2. Create a Lambda integration with EmployeeManagementFunction in ap-southeast-2.
3. Add the routes below and connect them to the Lambda integration.
4. Use $default stage with auto-deploy enabled.

Method	Route	Purpose
POST	/employees	Create employee

Method	Route	Purpose
GET	/employees	List employees
GET	/employees/{employeeId}	Get employee by ID
PUT	/employees/{employeeId}	Update employee
DELETE	/employees/{employeeId}	Delete employee

Step 6 — Test API using Postman

For POST, choose Body → raw → JSON:

```
POST /employees
{
  "employeeId": "EMP004",
  "employeeName": "Sourav",
  "department": "DevOps",
  "salary": 75000
}
```

For PUT /employees/EMP004, send:

```
{
  "department": "SRE",
  "salary": 80000
}
```

For DELETE /employees/EMP004, send a DELETE request with no body. Use GET /employees to verify records. During the API test sequence, EMP004 was created, updated and deleted. The remaining sample records were EMP001 (Rahul) and EMP003 (Barun).

5. Frontend Development and S3 Hosting

The frontend is a single index.html file containing HTML, CSS and JavaScript. JavaScript fetch() calls the API Gateway endpoints. The page supports listing, adding, editing and deleting employees.

API configuration in frontend

```
const API_BASE =
  "https://jndh8i2d68.execute-api.ap-southeast-2.amazonaws.com";
```

Frontend action	API request
Load employee list	GET /employees
Add employee	POST /employees
Save edit	PUT /employees/{employeeId}
Delete employee	DELETE /employees/{employeeId}

Step 7 — Create S3 bucket

1. Open Amazon S3 in ap-southeast-2.
2. Create a General purpose bucket with a globally unique name.
3. Keep ACLs disabled; versioning can remain disabled for this learning project.
4. Upload the frontend file to the bucket root as index.html.

Step 8 — Enable static website hosting

1. Open bucket → Properties.
2. Find Static website hosting and select Edit.
3. Enable hosting and choose Host a static website.
4. Set Index document to index.html.
5. Save changes.

Step 9 — Configure S3 website access

For the basic S3 website endpoint approach used in this learning project, public read access to website objects was enabled. This requires changing the bucket-level Block Public Access setting and adding a bucket policy granting s3:GetObject on objects in this bucket.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "PublicReadForWebsite",
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::YOUR_BUCKET_NAME/*"
  }]
}
```

Security warning: This is a learning setup. Do not store secrets, credentials or private data in this public bucket. For production, keep S3 private and use CloudFront with Origin Access Control (OAC).

Step 10 — Configure API Gateway CORS

The S3 website and API Gateway have different origins. Configure CORS on the HTTP API to allow browser JavaScript to call the API.

CORS setting	Learning project value
Allow-Origin	*
Allow-Headers	content-type
Allow-Methods	GET, POST, OPTIONS, PUT, DELETE

CORS setting	Learning project value
Max-Age	300 seconds
Allow-Credentials	No

Use the console Add control to add origin/header/method values where required. Save the configuration and deploy if needed. The $default stage was set to auto-deploy. In production, restrict allowed origins to the exact frontend domain.

Step 11 — Upload and open frontend

1. Upload index.html to the S3 bucket root, not inside another folder.
2. Open bucket Properties → Static website hosting.
3. Copy the Bucket website endpoint and open it in a browser.
4. Refresh with Ctrl+F5 after frontend changes.

6. End-to-End Testing

The project was tested through Postman and through the S3-hosted frontend.

Test	Verification	Result
GET list	Employee records load in UI	Successful
GET one	Employee returned by ID	Successful
POST	New employee saved	Successful
PUT	Employee details updated	Successful
DELETE	Test employee removed	Successful

At one point the verified sample records were EMP001 / Rahul / SRE / 60000 and EMP003 / Barun / SDE / 90000. During later frontend testing, EMP002 / Rakesh was also added and appeared in DynamoDB. Use the live DynamoDB table as the source of truth for current records.

7. GitHub Repository Structure

Recommended structure aligned to the existing local folders:

```
lambda-dynamodb/
|-- backend/
|   |-- lambda_function.py
|   |-- requirements.txt
|-- frontend_page/
|   |-- index.html
|-- diagram/
|   |-- architecture.png
|-- Documents/
|   |-- AWS_Employee_Management_Project_Guide.pdf
|-- README.md
|-- .gitignore
```

Keep the current folder names if they already contain files. Make sure the frontend file is named index.html. Check screenshots for account IDs, personal data and other sensitive details before committing.

.gitignore example

```
__pycache__/
*.pyc
.env
.env.*
.venv/
venv/
.vscode/
.idea/
*.log
terraform.tfstate
terraform.tfstate.backup
```

GitHub upload commands

```
# Run from the project root in PowerShell
git init
git status
git add .
git commit -m "Initial commit: AWS Employee Management app"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/lambda-dynamodb.git
git push -u origin main
```

Create the GitHub repository first. Replace YOUR_USERNAME and the repository URL. Review git status before committing; do not commit credentials, access keys or secret values.

8. Troubleshooting Guide

Symptom	Possible cause	What to inspect
S3 website 403	Public access/policy/object path	Static hosting enabled; index.html at bucket root; read policy.
Frontend loads but API fails	CORS or API request issue	CORS origin, content-type header, methods; browser Console/Network.
API 500	Lambda error or IAM issue	Lambda CloudWatch logs; role policy; table name and region.
Invalid action	Handler expects older event shape	Use HTTP API v2 method/path handling.
POST conditional error	Employee ID already exists	Use a unique ID or inspect the existing item.
GET one returns 404	Employee ID not found	Check exact employeeId partition key value.

Troubleshooting workflow

1. Record the method, URL, status code and error message.
2. Inspect browser DevTools → Console and Network.
3. Test the same endpoint in Postman to isolate frontend vs backend.
4. Inspect Lambda Monitor → CloudWatch Logs.
5. Check IAM permissions, DynamoDB table name and region.
6. Retest the affected route and the full browser workflow.

9. Production Readiness Recommendations

This project demonstrates the serverless CRUD workflow; it is not yet production-hardened. Before production use, consider:

• Private S3 bucket with CloudFront and Origin Access Control.
• Authentication and authorization, for example a suitable JWT/Cognito design.
• Restrict CORS to the real frontend origin.
• Validate request fields, lengths and salary values.
• API throttling and abuse protection.
• CloudWatch alarms for Lambda errors, throttles and API 5xx responses.
• DynamoDB Scan pagination for larger tables; Scan is not ideal for large-scale listing.
• Backups / point-in-time recovery and a recovery plan.
• Infrastructure as Code and controlled deployment automation when scope expands.

10. Interview Explanation

“I built a serverless Employee Management web application on AWS. I hosted the HTML, CSS and JavaScript frontend on Amazon S3. The frontend calls an API Gateway HTTP API, which invokes a Python Lambda function. Lambda uses Boto3 and an IAM execution role to perform CRUD operations on DynamoDB. I configured CORS for browser access and tested the API using Postman as well as the hosted frontend.”

Possible interview questions

1. Why Lambda? — Event-driven backend execution without managing servers.
2. Why API Gateway? — Managed HTTP endpoints and routing to Lambda.
3. Why DynamoDB? — Managed NoSQL storage suited to this simple employee record model.
4. Why IAM execution role? — Grants Lambda scoped temporary permissions to AWS services.
5. Why CORS? — Browsers enforce cross-origin rules when frontend and API use different origins.
6. Why not expose DynamoDB directly? — Backend logic and access control belong in the service layer, not in browser code.

11. Completion Checklist

- ■ DynamoDB EmployeeTable created
- ■ Lambda execution role and table permissions configured
- ■ EmployeeManagementFunction deployed
- ■ API Gateway HTTP API and five routes configured
- ■ Postman CRUD tests completed
- ■ S3 static website hosting enabled
- ■ index.html uploaded to bucket root
- ■ CORS configured and browser API calls verified
- ■ Frontend CRUD operations verified
- ■ README and .gitignore prepared
- ■ Repository checked for secrets before GitHub push

Document note: This guide records the project configuration and workflow discussed in the conversation. Confirm live AWS console values before reuse. The exact local contents of lambda_function.py and index.html were not independently inspected when this guide was generated.